

Recent Advancements in Natural Language Processing

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



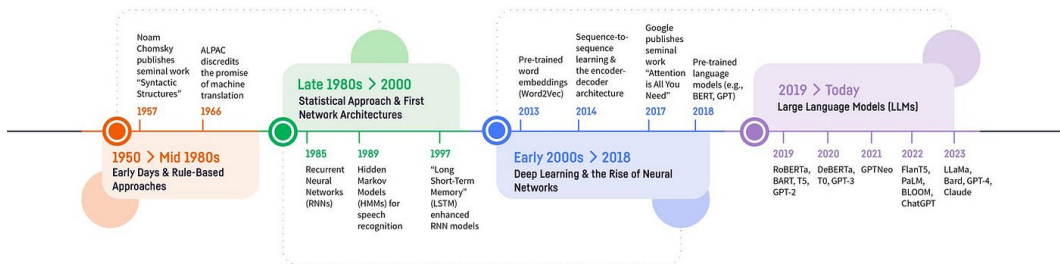
جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

The History of NLP



Source: Dataiku.

1. Motivation
2. Learning Outcomes
3. Inside FlashAttention
4. The LLaMA 3 Herd of Models
5. Byte Latent Transformer (BLT)
6. Large Language Diffusion Models (LLDMs)
7. Summary
8. References

Four departures from the standard recipe of an autoregressive transformer over subword tokens:

- ▶ **IO-aware kernels:** FlashAttention keeps exact attention but reorganizes its arithmetic around the GPU memory hierarchy, so the $N \times N$ attention matrix never touches slow memory.
- ▶ **Open-weight LLMs:** Releases like the LLaMA 3 herd bring frontier-level models into the open, widening access to research and deployment.
- ▶ **Tokenizer-free modeling:** the Byte Latent Transformer (BLT) replaces subword tokenization with learned byte patches.
- ▶ **Non-autoregressive generation:** Large Language Diffusion Models (LLDMs) generate text by iterative denoising instead of left-to-right prediction.

By the end of this session, you should be able to:

- ▶ Explain why attention is memory-bound rather than compute-bound, and how FlashAttention combines tiling, online softmax, and recomputation into exact IO-efficient attention.
- ▶ Describe the LLaMA 3 herd of models and what open-weight frontier models enable.
- ▶ Explain the Byte Latent Transformer (BLT): why tokenization is a bottleneck, and how entropy-based byte patching replaces it.
- ▶ Summarize Large Language Diffusion Models (LLDMs) and how diffusion-based text generation differs from autoregressive decoding.
- ▶ Situate these emerging paradigms relative to the standard transformer recipe and identify the trade-offs each one makes.

Inside FlashAttention

- ▶ Transformer training is usually **memory-bound**: matrix multiplications account for 99.8% of the FLOPs but only 61% of the runtime.
- ▶ Nearly 40% of the time goes to softmax, dropout, masking, and normalization — operations that do almost no arithmetic but stream whole tensors through GPU memory.
- ▶ Recall the hierarchy from the FlashAttention-2 discussion: roughly 20 MB of on-chip SRAM at 19 TB/s against 40–80 GB of HBM at 1.5–2 TB/s.
- ▶ So the lever for faster attention is not fewer FLOPs; it is **less data movement**.

Operator class	% FLOPs	% runtime
Tensor contraction	99.80	61.0
Stat. normalization	0.17	25.5
Element-wise	0.03	13.5

Operator classes in a PyTorch BERT training step. Ivanov et al., “Data Movement Is All You Need: A Case Study on Optimizing Transformers,” [arXiv:2007.00072](https://arxiv.org/abs/2007.00072), Table 1.

The textbook implementation materializes two $N \times N$ matrices in slow memory:

1. Load Q, K by blocks from HBM, compute $S = QK^\top$, **write S to HBM**.
 2. Read S from HBM, compute $P = \text{softmax}(S)$, **write P to HBM**.
 3. Load P and V by blocks from HBM, compute $O = PV$, write O to HBM.
- At $N = 4096$ in fp16, one head's score matrix is already $4096^2 \times 2 \text{ B} = 32 \text{ MiB}$ — larger than *all* the SRAM on the GPU — while the Q tile it came from is half a MiB.
 - Every step is a separate kernel: the $N \times N$ traffic is paid twice for S and twice more for P , and again during the backward pass, which needs both matrices.

Standard attention implementation, Algorithm 0 of Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” [arXiv:2205.14135](https://arxiv.org/abs/2205.14135).

Split the matrices into blocks; each output block is a short sum of block products:

$$C = \begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix}, \quad C_{ij} = \sum_k A_{ik} B_{kj}, \quad C_{00} = A_{00}B_{00} + A_{01}B_{10} + A_{02}B_{20}.$$

- ▶ With three tile-sized registers X, Y, Z in fast memory, C_{00} never leaves the chip until it is finished:

$$\begin{aligned} X &\leftarrow A_{00}, & Y &\leftarrow B_{00}, & Z &\leftarrow XY, \\ X &\leftarrow A_{01}, & Y &\leftarrow B_{10}, & Z &\leftarrow Z + XY, \\ X &\leftarrow A_{02}, & Y &\leftarrow B_{20}, & Z &\leftarrow Z + XY, & C_{00} &\leftarrow Z. \end{aligned}$$

- ▶ Each input tile is read once per output block, and the big matrices live in slow memory only.
- ▶ The trick works because **addition is associative**: partial products can be accumulated in any order, one pair of tiles at a time. Every fast GEMM library is built on this.

$$X' = \text{softmax}\left(QK^{\top} / \sqrt{d_k}\right) V$$

- ▶ The matrix products in attention tile perfectly — but between them sits a row-wise softmax, and **softmax is not associative**.
- ▶ Each output row needs the row's *global* maximum and normalizer:

$$y_i = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}, \quad m = \max_k x_k,$$

so no block of the output can be finalized until every block of the row has been seen.

- ▶ Tile the two matmuls naively and the softmax in the middle still forces the full $N \times N$ score matrix out to HBM — which is exactly the traffic tiling was meant to remove.

- ▶ The naive form $y_i = e^{x_i} / \sum_j e^{x_j}$ overflows quickly: e^{89} exceeds the fp32 range, and e^{12} already exceeds fp16.
- ▶ Every deep-learning library therefore computes the **safe softmax**, shifted by the maximum:

$$\text{pass 1: } m = \max_k x_k$$

$$\text{pass 2: } d = \sum_{j=1}^V e^{x_j - m}$$

$$\text{pass 3: } y_i = e^{x_i - m} / d$$

- ▶ Numerically robust — but it makes **three full passes** over the logits, and for large inputs each pass is a full trip through memory.

Milakov & Gimelshein, “Online normalizer calculation for softmax,” [arXiv:1805.02867](https://arxiv.org/abs/1805.02867), Algorithms 1–2.

- ▶ The running maximum and the running normalizer can be maintained **in the same pass**. When a new element x_j arrives:

$$m_j = \max(m_{j-1}, x_j)$$
$$d_j = d_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}$$

- ▶ The factor $e^{m_{j-1} - m_j}$ **retroactively rescales** everything accumulated so far, as if the new maximum had been known from the start; after one pass, $y_i = e^{x_i - m_v} / d_v$ needs only one more.
- ▶ Three passes become two: a $1.33\times$ saving on paper and about $1.3\times$ in practice, purely from reduced memory traffic — the arithmetic went *up*.
- ▶ This recurrence, published as a softmax micro-optimization, turned out to be the missing piece that lets attention be tiled.

Operator Fusion: Skip the Round Trips



- Unfused (left), every memory-bound op — scale, mask, softmax, dropout — pays its own HBM round trip; fused (right), the whole chain rides along in a single kernel in fast memory.
- Fusing *across* attention's matmuls is exactly what the softmax normalizer forbids — unless the softmax itself can be computed incrementally.

Figures from Horace He, "Making Deep Learning Go Brrrr From First Principles,"

https://horace.io/brrr_intro.html.

Tile K , V into blocks; keep running statistics (m, ℓ) and output O per query row. For each incoming block j , compute on-chip

$$S_j = QK_j^\top / \sqrt{d_k}, \quad m_j = \text{rowmax}(S_j), \quad P_j = e^{S_j - m_j}, \quad \ell_j = \text{rowsum}(P_j),$$

then fold it into the running answer with the online-softmax correction:

$$m' = \max(m, m_j), \quad \ell' = e^{m - m'} \ell + e^{m_j - m'} \ell_j, \\ O' = \frac{\ell e^{m - m'}}{\ell'} O + \frac{e^{m_j - m'}}{\ell'} P_j V_j.$$

- ▶ This is the online-softmax recurrence upgraded from a running sum to a running *weighted sum of value rows*: the $N \times N$ matrix is never materialized, and the result is **exact**.
- ▶ FlashAttention-2's “fewer non-matmul FLOPs” improvement is precisely a deferral of the division by ℓ to the very end of the loop.

- ▶ The backward pass needs the attention matrices S and P — but storing them would reintroduce the $N \times N$ traffic.
- ▶ FlashAttention stores only the output O and the softmax statistics (m, ℓ) , of size N , and **recomputes** the needed tiles of S and P on-chip during the backward pass.
- ▶ The result does *more* arithmetic than standard attention, moves $9\times$ less data, and finishes almost $6\times$ sooner — the memory-bound argument, closed in one table.

	Standard	Flash
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

Forward + backward of one attention layer, GPT-2 medium on an A100. Dao et al., [arXiv:2205.14135](#),

Figure 2.

- ▶ **No approximation:** the model and its perplexity are unchanged; only time and memory move. GPT-2 medium trains in 6.9 days against 21.0 for the baseline implementation, at identical quality.
- ▶ **Linear memory:** the footprint grows with N instead of N^2 , which is what made 16k-token Path-X tractable — the first Transformer to beat chance on it.
- ▶ **Hardware–software co-design:** the algorithm is reorganized around the memory hierarchy, and each GPU generation reshapes the kernel — the engineering that FlashAttention-2 carries further (see the *Transformers: 2017 vs 2026* lecture).
- ▶ **Long context as the payoff:** IO-efficient exact attention is the entry ticket to the 128k-token windows of models like LLaMA 3, and to the long interactive sessions that agentic workloads demand; the serving-side levers (KV caching, paging, speculative decoding) are covered in the *Inference Optimisation* lecture.

The LLaMA 3 Herd of Models



The Llama 3 Herd of Models

Llama Team, AI @ Meta¹

¹A detailed contributor list can be found in the appendix of this paper.

Modern artificial intelligence (AI) systems are powered by foundation models. This paper presents a new set of foundation models, called Llama 3. It is a herd of language models that natively support multilinguality, coding, reasoning, and tool usage. Our largest model is a dense Transformer with 405B parameters and a context window of up to 128K tokens. This paper presents an extensive empirical evaluation of Llama 3. We find that Llama 3 delivers comparable quality to leading language models such as GPT-4 on a plethora of tasks. We publicly release Llama 3, including pre-trained and post-trained versions of the 405B parameter language model and our Llama Guard 3 model for input and output safety. The paper also presents the results of experiments in which we integrate image, video, and speech capabilities into Llama 3 via a compositional approach. We observe this approach performs competitively with the state-of-the-art on image, video, and speech recognition tasks. The resulting models are not yet being broadly released as they are still under development.

Date: July 23, 2024

Website: <https://llama.meta.com/>

Source: Llama Team, AI @ Meta, "The Llama 3 Herd of Models," [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), title page.

Meta AI (2024)

- ▶ **Model Sizes:** 8B, 70B, and 405B
- ▶ **Open Weights:** Available for commercial use
- ▶ **Training:** The 405B model is pre-trained on 15.6 trillion multilingual tokens, against 1.8 trillion for LLaMA 2
- ▶ **Architecture:** A standard dense transformer; the gains come from data quality and training scale rather than architectural novelty

The LLaMA 3 Herd of Models (cont.)

	Finetuned	Multilingual	Long context	Tool use	Release
Llama 3 8B	✗	✗ ¹	✗	✗	April 2024
Llama 3 8B Instruct	✓	✗	✗	✗	April 2024
Llama 3 70B	✗	✗ ¹	✗	✗	April 2024
Llama 3 70B Instruct	✓	✗	✗	✗	April 2024
Llama 3.1 8B	✗	✓	✓	✗	July 2024
Llama 3.1 8B Instruct	✓	✓	✓	✓	July 2024
Llama 3.1 70B	✗	✓	✓	✗	July 2024
Llama 3.1 70B Instruct	✓	✓	✓	✓	July 2024
Llama 3.1 405B	✗	✓	✓	✗	July 2024
Llama 3.1 405B Instruct	✓	✓	✓	✓	July 2024

Overview of the Llama 3 Herd of models. All results in this paper are for the Llama 3.1 models.

The April 2024 release (8B, 70B) is short-context and was intended for English use at the time; the multilingual, long-context, tool-using models arrive with LLaMA 3.1 in July 2024, and every result in the paper is for those. Source: Llama Team, AI @ Meta, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Table 1.

Key Features:

- ▶ Context window of up to 128k tokens, reached in two stages: pre-training at 8k, then continued pre-training on longer sequences
- ▶ Grouped-query attention with 8 key-value heads, which shrinks the key-value cache and speeds up decoding
- ▶ A 128k-token vocabulary: 100k from the `tiktoken` tokenizer plus 28k added for non-English languages
- ▶ FP8 quantization of the feed-forward layers for low-precision inference on H100 GPUs; the self-attention layers are left unquantized
- ▶ Released fine-tuned variants are the Instruct models

The LLaMA 3 Herd of Models (cont.)

Category	Benchmark	Llama 3 8B	Gemma 2 9B	Mistral 7B	Llama 3 70B	Mixtral 8x22B	GPT 3.5 Turbo	Llama 3 405B	Nemotron 4 340B	GPT-4o (o1.5)	GPT-4o	Claude 3.5 Sonnet
General	MMLU (5-shot)	69.4	72.3	61.1	83.6	76.9	70.7	87.3	82.6	85.1	89.1	89.9
	MMLU (0-shot, CoT)	73.0	72.3 [△]	60.5	86.0	79.9	69.8	88.6	78.7 [◊]	85.4	88.7	88.3
	MMLU-Pro (5-shot, CoT)	48.3	–	36.9	66.4	56.3	49.2	73.3	62.7	64.8	74.0	77.0
	IFEval	80.4	73.6	57.6	87.5	72.7	69.9	88.6	85.1	84.3	85.6	88.0
Code	HumanEval (0-shot)	72.6	54.3	40.2	80.5	75.6	68.0	89.0	73.2	86.6	90.2	92.0
	MBPP EvalPlus (0-shot)	72.8	71.7	49.5	86.0	78.6	82.0	88.6	72.8	83.6	87.8	90.5
Math	GSM8K (8-shot, CoT)	84.5	76.7	53.2	95.1	88.2	81.6	96.8	92.3 [◊]	94.2	96.1	96.4 [◊]
	MATH (0-shot, CoT)	51.9	44.3	13.0	68.0	54.1	43.1	73.8	41.1	64.5	76.6	71.1
Reasoning	ARC Challenge (0-shot)	83.4	87.6	74.2	94.8	88.7	83.7	96.9	94.6	96.4	96.7	96.7
	GPQA (0-shot, CoT)	32.8	–	28.8	46.7	33.3	30.8	51.1	–	41.4	53.6	59.4
Tool use	BFCL	76.1	–	60.4	84.8	–	85.9	88.5	86.5	88.3	80.5	90.2
	Nexus	38.5	30.0	24.7	56.7	48.5	37.2	58.7	–	50.3	56.1	45.7
Long context	ZeroSCROLLS/QuALITY	81.0	–	–	90.5	–	–	95.2	–	95.2	90.5	90.5
	InfiniteBench/En.MC	65.1	–	–	78.2	–	–	83.4	–	72.1	82.5	–
	NIH/Multi-needle	98.8	–	–	97.5	–	–	98.1	–	100.0	100.0	90.8
Multilingual	MGSM (0-shot, CoT)	68.9	53.2	29.9	86.9	71.1	51.4	91.6	–	85.9	90.5	91.6

Table 2 Performance of finetuned Llama 3 models on key benchmark evaluations. The table compares the performance of the 8B, 70B, and 405B versions of Llama 3 with that of competing models. We **boldface** the best-performing model in each of three model-size equivalence classes. [△]Results obtained using 5-shot prompting (no CoT). [◊]Results obtained without CoT. [◊]Results obtained using zero-shot prompting.

Source: Llama Team, AI @ Meta, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Table 2.

The LLaMA 3 Herd of Models (cont.)

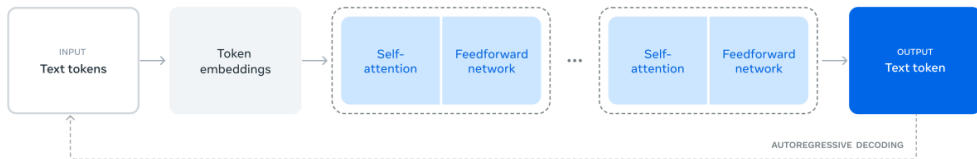


Figure 1 Illustration of the overall architecture and training of Llama 3. Llama 3 is a Transformer language model trained to predict the next token of a textual sequence. See text for details.

Source: Llama Team, AI @ Meta, [arXiv:2407.21783](#), Figure 1.

The LLaMA 3 Herd of Models (cont.)

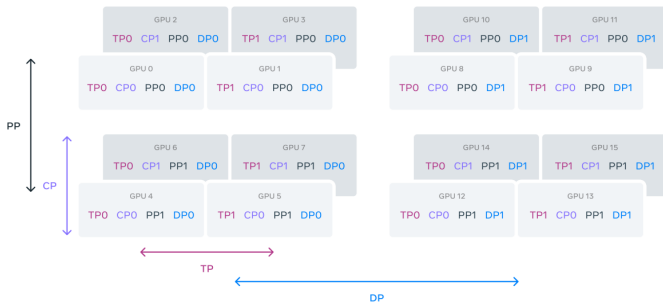


Figure 5 Illustration of 4D parallelism. GPUs are divided into parallelism groups in the order of [TP, CP, PP, DP], where DP stands for FSDP. In this example, 16 GPUs are configured with a group size of $|TP|=2$, $|CP|=2$, $|PP|=2$, and $|DP|=2$. A GPU's position in 4D parallelism is represented as a vector, $[D_1, D_2, D_3, D_4]$, where D_i is the index on the i -th parallelism dimension. In this example, GPU0[TP0, CP0, PP0, DP0] and GPU1[TP1, CP0, PP0, DP0] are in the same TP group, GPU0 and GPU2 are in the same CP group, GPU0 and GPU4 are in the same PP group, and GPU0 and GPU8 are in the same DP group.

Training the 405B model splits each batch across four parallelism dimensions at once: tensor, context, pipeline, and data.
Source: Llama Team, AI @ Meta, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Figure 5.

The LLaMA 3 Herd of Models (cont.)

User prompt

Based on the inflation rate for the US in February 2024, and the cost of this list of tuples containing my daily groceries, `[("eggs", 1.1), ("chicken", 4.0), ("rice", 2.0), ("beans", 2.5), ("apples", 6.25)]`, can you give me the estimated total cost of my groceries over a 30 day month when you add 1 cookie per day at Costco's food court giant cookie debut price and adjust the total for the inflation percentage in \$X.XX?

System Prompt

Environment: `python`,
Tools: `brave_search`, `wolfram_alpha`
Cutting Knowledge Date: `December 2023`
Today's Date: `Jul 11, 2024`

Assistant response

To answer your question, I will need to:

- Look up the inflation rate for the US in February 2024
- Look up the Costco food court giant cookie debut price
- Calculate the estimated total cost of your groceries over a 30 day month when you add 1 cookie per day at Costco's food court giant cookie debut price and adjust the total for the inflation percentage

First, I will look up the inflation rate for the US in February 2024.

Assistant tool call

`brave_search.call(query="US Inflation rate February 2024")`

Tool output

`{ "title": "US Inflation Remained Higher Than Economists Had Expected in February", "url": "https://www.investopedia.com/february-2024-cpi-6087020/", "description": "Consumer prices rose 0.2% over the year in February, higher than the 3.1% annual rate in January and above what economists had predicted. An uptick in gas prices and housing prices was behind the unexpectedly high inflation rate." }`

The inflation rate for the US in February 2024 was 3.2%. Next, I will look up the Costco food court giant cookie debut price.

Assistant tool call

`brave_search.call(query="Costco food court giant cookie debut price")`

Tool output

`{ "title": "Costco's food court replaced its churro with a giant cookie. Here's how it tastes", "url": "https://www.today.com/food/groceries/costco-double-chocolate-chunk-cookie-food-court-revealed-rcna105317", "description": "Costco's Kirkland Signature Double Chocolate Chunk cookie is the latest addition to its food court, costs $2.49 and has 750 calories. Here's a my review of the buttery treat." }`

The Costco food court giant cookie debut price is \$2.49. Now, I will calculate the estimated total cost of your groceries over a 30 day month when you add 1 cookie per day at Costco's food court giant cookie debut price and adjust the total for the inflation percentage.

Assistant tool call

Define the list of tuples containing the daily groceries
`groceries = [("eggs", 1.1), ("chicken", 4.0), ("rice", 2.0), ("beans", 2.5), ("apples", 6.25)]`

Calculate the total cost of the groceries per day
`total_cost_per_day = sum([item[1] for item in groceries])`

Calculate the total cost of the groceries per day including the cookie
`total_cost_per_day_with_cookie = total_cost_per_day + 2.49`

Calculate the total cost of the groceries for 30 days including the cookie
`total_cost_for_30_days_with_cookie = total_cost_per_day_with_cookie * 30`

Define the inflation rate
`inflation_rate = 0.032`

Calculate the total cost of the groceries for 30 days including the cookie and adjusted for inflation
`total_cost_for_30_days_with_cookie_and_inflation = total_cost_for_30_days_with_cookie * (1 + inflation_rate)`

Print the result
`print(f"The estimated total cost of your groceries over a 30 day month when you add 1 cookie per day at Costco's food court giant cookie debut price and adjust the total for the inflation percentage is ${total_cost_for_30_days_with_cookie_and_inflation:.2f}").`

Tool output

The estimated total cost of your groceries over a 30 day month when you add 1 cookie per day at Costco's food court giant cookie debut price and adjust the total for the inflation percentage is **\$626.63**.

Figure 10 Multi-step tool usage. Example of Llama 3 performing multi-step planning, reasoning, and tool calling to solve a task.

Source: Llama Team, AI @ Meta, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Figure 10.

KAUST Academy

Recent Advancements in NLP

25

	Reading Comprehension		
	SQuAD	QuAC	RACE
Llama 3 8B	77.0 $\pm$ 0.8	44.9 $\pm$ 1.1	54.3 $\pm$ 1.4
Mistral 7B	73.2 $\pm$ 0.8	44.7 $\pm$ 1.1	53.0 $\pm$ 1.4
Gemma 7B	81.8 $\pm$ 0.7	42.4 $\pm$ 1.1	48.8 $\pm$ 1.4
Llama 3 70B	81.8 $\pm$ 0.7	51.1 $\pm$ 1.1	59.0 $\pm$ 1.4
Mixtral 8 $\times$ 22B	84.1 $\pm$ 0.7	44.9 $\pm$ 1.1	59.2 $\pm$ 1.4
Llama 3 405B	81.8 $\pm$ 0.7	53.6 $\pm$ 1.1	58.1 $\pm$ 1.4
GPT-4	—	—	—
Nemotron 4 340B	—	—	—
Gemini Ultra	—	—	—

Table 9 Pre-trained model performance on reading comprehension tasks. Results include 95% confidence intervals.

	Code	
	HumanEval	MBPP
Llama 3 8B	37.2 $\pm$ 7.4	47.6 $\pm$ 4.4
Mistral 7B	30.5 $\pm$ 7.0	47.5 $\pm$ 4.4
Gemma 7B	32.3 $\pm$ 7.2	44.4 $\pm$ 4.4
Llama 3 70B	58.5 $\pm$ 7.5	66.2 $\pm$ 4.1
Mixtral 8 $\times$ 22B	45.1 $\pm$ 7.6	71.2 $\pm$ 4.0
Llama 3 405B	61.0 $\pm$ 7.5	73.4 $\pm$ 3.9
GPT-4	67.0 $\pm$ 7.2	—
Nemotron 4 340B	57.3 $\pm$ 7.6	—
Gemini Ultra	74.4 $\pm$ 6.7	—

Table 10 Pre-trained model performance on coding tasks. Results include 95% confidence intervals.

Source: Llama Team, AI @ Meta, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Tables 9 and 10.

Highlights:

- ▶ Quality comparable to leading closed models such as GPT-4 across a broad benchmark suite
- ▶ Image, video, and speech capabilities are added compositionally: adapters and cross-attention layers are trained on top of the language model while its weights stay frozen, so text-only quality is preserved
- ▶ Those multimodal variants were still under development, and not released, at publication

Byte Latent Transformer (BLT)

Byte Latent Transformer: Patches Scale Better Than Tokens

Artidoro Pagnoni, Ram Pasunuru[†], Pedro Rodriguez[‡], John Nguyen[†], Benjamin Muller, Margaret Li^{1,◊},
Chunting Zhou[◊], Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman^{†,2,◊},
Srinivasan Iyer[†]

FAIR at Meta, ¹Paul G. Allen School of Computer Science & Engineering, University of Washington,

²University of Chicago

[‡]Joint second author, [†]Joint last author, [◊]Work done at Meta

We introduce the Byte Latent Transformer (BLT), a new byte-level LLM architecture that, for the first time, matches tokenization-based LLM performance at scale with significant improvements in inference efficiency and robustness. BLT encodes bytes into dynamically sized patches, which serve as the primary units of computation. Patches are segmented based on the entropy of the next byte, allocating more compute and model capacity where increased data complexity demands it. We present the first FLOP controlled scaling study of byte-level models up to 8B parameters and 4T training bytes. Our results demonstrate the feasibility of scaling models trained on raw bytes without a fixed vocabulary. Both training and inference efficiency improve due to dynamically selecting long patches when data is predictable, along with qualitative improvements on reasoning and long tail generalization. Overall, for fixed inference costs, BLT shows significantly better scaling than tokenization-based models, by simultaneously growing both patch and model size.

Date: December 16, 2024

Correspondence: artidoro at cs.washington.edu, sviyer at meta.com

Code: <https://github.com/facebookresearch/blt>



Source: Pagnoni et al., “Byte Latent Transformer: Patches Scale Better Than Tokens,” [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), title page.

BLT is a byte-level LLM with no tokenizer and no fixed vocabulary:

- ▶ Raw bytes are grouped into **dynamically sized patches**; one patch, rather than one token, is one step of the large model.
- ▶ Patch boundaries come from the entropy of the next byte under a small byte-level language model, so long patches form where the text is predictable and short ones where it is not.
- ▶ Inference cost is inversely proportional to the average patch size, which turns patch size into a scaling axis of its own: the model can grow while the inference budget stays fixed.
- ✓ Matches LLaMA 3 in a FLOP-controlled comparison up to 8B parameters and 4T training bytes
- ✓ Up to 50% fewer inference FLOPs, by trading small losses on evaluation metrics
- ✓ Stronger character-level understanding and better robustness to noisy input

Byte Latent Transformer (BLT) (cont.)

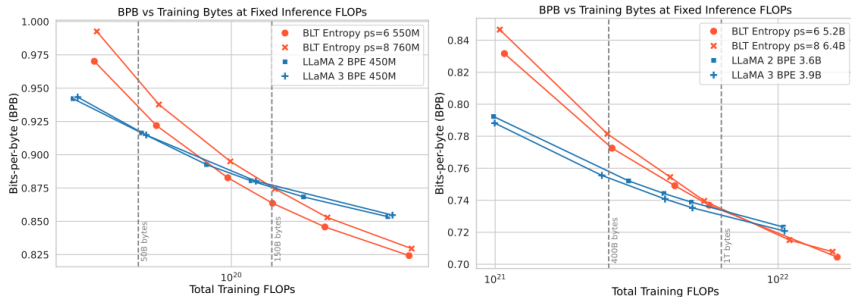


Figure 1 Scaling trends for fixed inference FLOP models (fully) trained with varying training budgets. In token-based models, a fixed inference budget determines the model size. In contrast, the BLT architecture provides a new scaling axis allowing simultaneous increases in model and patch size while keeping the same training and inference budget. BLT patch-size (ps) 6 and 8 models quickly overtake scaling trends of BPE Llama 2 and 3. Moving to the larger inference budget makes the larger patch size 8 model more desirable sooner. Both BPE compute-optimal point and crossover point are indicated with vertical lines.

Quality is measured in bits-per-byte, which stays comparable across models that segment text differently, unlike per-token perplexity. Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 1.

Byte Latent Transformer (BLT) (cont.)

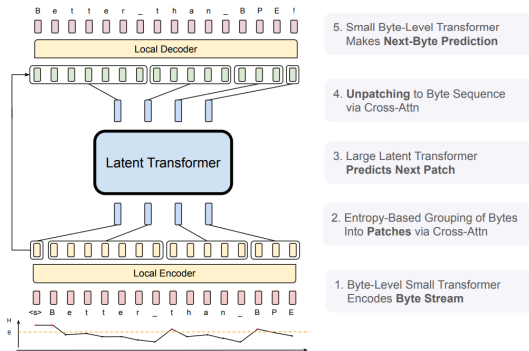


Figure 2 BLT comprises three modules, a lightweight *Local Encoder* that encodes input bytes into patch representations, a computationally expensive Latent Transformer over patch representations, and a lightweight *Local Decoder* to decode the next patch of bytes. BLT incorporates byte n -gram embeddings and a cross-attention mechanism to maximize information flow between the Latent Transformer and the byte-level modules (Figure 5). Unlike fixed-vocabulary tokenization, BLT dynamically groups bytes into patches preserving access to the byte-level information.

ByT5, covered earlier, gave up patching altogether and paid for attention over every byte. Here the expensive module runs once per patch, and the byte-level work is confined to two small ones. Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 2.

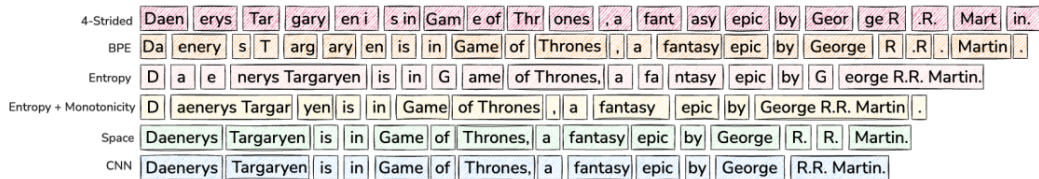


Figure 3 Patching schemes group bytes in different ways, each leading to a different number of resulting patches. Since each patch is processed using a large transformer step, the number of patches directly determines the bulk of the compute expended in terms of FLOPs. These schemes group bytes into patches by (a) striding every four bytes (§2.1) as in MegaByte (Yu et al., 2023), (b) tokenizing with Byte-Pair Encoding (BPE), in this case the Llama-3 (Dubey et al., 2024) tokenizer, (c & d) entropy-based patching as in this work (§2.3), (e) patching on space-bytes (Slagle, 2024), (f) and patching on entropy using a small CNN byte-level model with 2-byte context.

The same sentence under six patching schemes; the number of patches sets the compute the large model spends. Row 1 is MEGABYTE’s fixed four-byte stride, row 2 the LLaMA 3 BPE tokenizer, rows 3–4 BLT’s entropy patching, row 5 SpaceByte’s split after space-like bytes. Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 3.

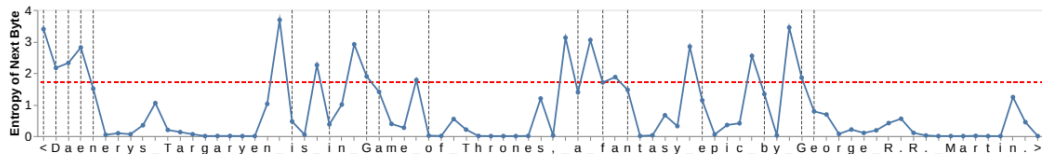


Figure 4 This figure plots the entropy $H(x_i)$ of each byte in “Daenerys Targaryen is in Game of Thrones, a fantasy epic by George R.R. Martin.” with spaces shown as underscores. Patches end when $H(x_i)$ exceeds the global threshold θ_g , shown as a red horizontal line. The start of new patches are shown with vertical gray lines. For example, the entropies of “G” and “e” in “George R.R. Martin” exceed θ_g , so “G” is the start of a single byte patch and “e” of a larger patch extending to the end of the named entity as the entropy $H(x_i)$ stays low, resulting in no additional patches.

Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 4.

Space Patching (Slagle, 2024) improves over strided patching by creating new patches after any space-like byte, which often aligns with word boundaries in many languages.

- ▶ Allocates a latent transformer step for every word.
- ▶ Ensures consistent patching of words across sequences.
- ▶ Focuses compute (flops) on hard predictions, typically following spaces.

Example: Predicting the first byte of the answer to “Who composed the Magic Flute?” is harder than predicting subsequent bytes in “Mozart”.

- ▶ The first character narrows down possible completions, making the rest easier to predict.

Limitations:

- ▶ Not suitable for all languages or domains.
- ▶ Cannot vary patch size.

Next: A new patching method that uses the difficulty of predicting initial bytes in words and enables control over patch size.

Entropy Patching: Using Next-Byte Entropies from a Small Byte LM

- ▶ Instead of rule-based heuristics (e.g., whitespace), entropy patching uses a data-driven approach to identify high-uncertainty next-byte predictions.
- ▶ Train a small byte-level auto-regressive LM on BLT training data.
- ▶ Compute next-byte entropies $H(x_i)$ under the LM distribution p_e over the byte vocabulary V :

$$H(x_i) = - \sum_{v \in V} p_e(x_i = v \mid x_{<i}) \log p_e(x_i = v \mid x_{<i}) \quad (1)$$

- ▶ Two methods for identifying patch boundaries:
 1. **Global Constraint:** $H(x_t) > \theta_g$ (entropy above a global threshold)
 2. **Approx. Monotonic Constraint:** $H(x_t) - H(x_{t-1}) > \theta_r$ (entropy rises relative to previous byte)
- ▶ The second method finds points that break approximate monotonically decreasing entropy within a patch.
- ▶ Patch boundaries are identified during lightweight preprocessing at dataloading.
- ▶ Differs from Nawrot et al. (2023), which trains a classifier to predict entropy-based patch boundaries.

In experiments: Both methods are compared for distinguishing between low and high entropy bytes.

Encoder Multi-Headed Cross-Attention:

- ▶ Follows Perceiver input cross-attention (Jaegle et al., 2021), but latent representations correspond to variable patch representations (not fixed set).
- ▶ Each patch p_j attends only to its own bytes.
- ▶ Query vector for each patch is initialized by pooling byte representations for p_j , then projecting via $EC \in \mathbb{R}^{h_E \times (h_E \times U_E)}$, where U_E is the number of encoder cross-attention heads.

Formulation:

$$P_{0,j} = EC(f_{\text{bytes}}(p_j)) \quad (\text{Pooling function } f \text{ over bytes in patch } p_j) \quad (2)$$

$$P_l = P_{l-1} + W_o \left[\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \right] \quad (3)$$

$$Q_j = W_q(P_{l-1,j}) \quad (4)$$

$$K_i = W_k(h_{l-1,i}) \quad (5)$$

$$V_i = W_v(h_{l-1,i}) \quad (6)$$

$$h_l = \text{Encoder-Transformer-Layer}_l(h_{l-1}) \quad (7)$$

- ▶ $P \in \mathbb{R}^{n_p \times h_G}$: n_p patch representations for the global model.
- ▶ W_q, W_k, W_v, W_o : learned projection matrices.

- Patch representations are initialized by pooling byte embeddings e_i for each patch p_j .

Key Difference: Latent representations are variable and correspond to patch boundaries, not a fixed latent set.

Encoder Multi-Headed Cross-Attention (cont.)

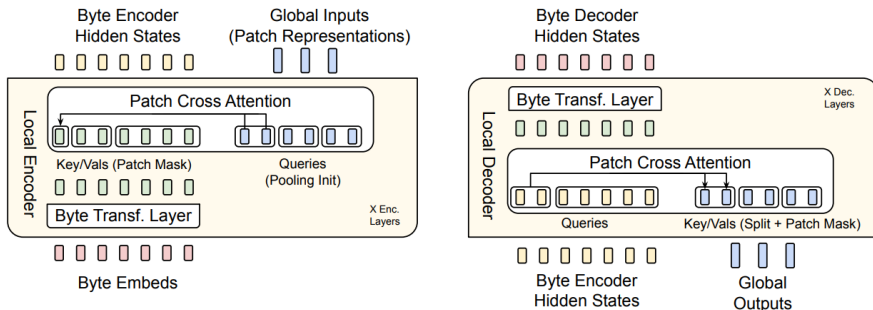


Figure 5 The local encoder uses a cross-attention block with patch representations as queries, and byte representations as keys/values to encode byte representations into patch representations. The local decoder uses a similar block but with the roles reversed i.e. byte representations are now the queries and patch representations are the keys/values. Here we use Cross-Attn $k = 2$.

Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 5.

Decoder Multi-Headed Cross-Attention:

- ▶ In the decoder, the roles of queries and key/values are swapped compared to the encoder cross-attention.
- ▶ Byte representations act as queries, while patch representations serve as keys and values.
- ▶ Initial byte representations for cross-attention are set as the byte embeddings from the last encoder layer, h^E .

Formulation:

$$D_0 = h^E \quad (8)$$

$$B_l = D_{l-1} + W_o \left[\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \right] \quad (9)$$

$$Q_i = W_q(d_{l-1,i}) \quad (10)$$

$$K_j = W_k(DC(o_j)) \quad (11)$$

$$V_j = W_v(DC(o_j)) \quad (12)$$

$$D_l = \text{Decoder-Transformer-Layer}_l(B_l) \quad (13)$$

- ▶ DC is a linear transformation and split operation applied to the final patch representations o_j from the global model.
- ▶ W_q, W_k, W_v, W_o are learned projection matrices.

- ▶ $B \in \mathbb{R}^{h_D \times n_b}$, where n_b is the number of output bytes.
- ▶ Multiple attention heads, pre-LayerNorm, no positional embeddings, and residual connections are used.

Summary: Decoder cross-attention enables each output byte to attend to patch-level latent representations, facilitating efficient and flexible decoding.

BLT Performance:

- ▶ **Parity at scale:** an 8B BLT trained on 4T bytes matches an 8B LLaMA 3 under a FLOP-controlled comparison, and reaches up to 50% fewer inference FLOPs when patch size is raised.
- ▶ **Character-level tasks:** on the CUTE benchmark of orthographic and sequence-manipulation tasks, BLT beats both BPE LLaMA 3 baselines by more than 25 points and scores 99.9% on both spelling tasks, despite seeing $16\times$ less training data than LLaMA 3.1. Word deletion and insertion are the only two CUTE tasks where the tokenizer-based model wins.
- ▶ **Low-resource translation:** on FLORES-101, BLT gains 2 BLEU points overall translating into English and 0.5 translating out of it. On the six widely used languages the two models trade places; the gains come from the twenty-one lower-resource languages.

Language	Language → English		English → Language	
	Llama 3	BLT	Llama 3	BLT
Arabic	22.3	24.6	10.4	8.8
German	41.3	42.0	29.8	31.2
Hindi	20.7	20.9	7.8	7.2
Italian	34.0	33.9	24.4	26.2
Vietnamese	31.2	31.0	28.4	23.7
Thai	17.9	18.1	10.5	7.7
Armenian	1.7	6.3	0.6	0.9
Amharic	1.3	3.1	0.4	0.5
Assamese	2.7	5.4	0.8	1.6
Bengali	4.7	12.7	1.7	4.1
Bosnian	36.0	37.3	16.9	19.6
Cebuano	18.2	20.6	5.8	9.1
Georgian	1.7	7.4	1.0	2.5
Gujarati	2.0	5.8	1.0	2.2
Hausa	5.75	5.9	1.2	1.3
Icelandic	16.1	17.9	4.8	5.3
Kannada	1.6	3.9	0.7	1.7
Kazakh	5.6	7.0	1.0	2.6
Kabuverdianu	20.3	20.9	5.1	6.8
Khmer	4.4	9.5	0.8	0.8
Kyrgyz	4.6	5.1	0.9	2.0
Malayalam	1.8	3.5	0.7	1.4
Odia	1.6	2.7	0.8	1.1
Somali	5.0	5.0	1.1	1.4
Swahili	10.1	12.0	1.4	2.3
Urdu	9.3	9.5	2.0	1.4
Zulu	4.7	5.0	0.6	0.5
Overall Average	12.1	14.0	5.9	6.4

Table 4 Performance of 8B BLT and 8B Llama 3 trained for 1T tokens on translating into and from six widely-used languages and twenty one lower resource languages with various scripts from the FLORES-101 benchmark (Goyal et al., 2022).

SentencePiece BLEU on FLORES-101. Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Table 4.

Task	Prompt	Llama 3	BLT
Substitute Word	Question: Substitute " and " with " internet " in " She went to the kitchen and saw two cereals. ". Answer:	She went to the kitchen and saw two cereals.	She went to the kitchen internet saw two cereals.
Swap Char	Question: Swap " h " and " a " in " that ". Answer:	that	taht
Substitute Char	Question: Substitute " a " with " m " in " page ". Answer:	-	pmge
Semantic Similarity	Question: More semantically related to " are ": " seem ", " acre ". Answer:	acre	seem
Orthographic Similarity	Question: Closer in Levenshtein distance to " time ": " timber ", " period ". Answer:	period	timber
Insert Char	Question: Add an " z " after every " n " in " not ". Answer:	znatz	nzot

Figure 7 Output responses from Llama 3 and BLT models for various tasks from CUTE benchmark. BLT model performs better on sequence manipulation tasks compared to the tokenizer-based Llama 3 model. Note that few-shot examples are not shown in the above prompts to maintain clarity.

Tasks a tokenizer hides from the model: a BPE model has no direct access to the characters inside its own tokens. Source: Pagnoni et al., [arXiv:2412.09871](https://arxiv.org/abs/2412.09871), Figure 7.

The tokenizer is removed, but not for free:

- ▶ **An extra model in the pipeline.** Segmentation needs a separately trained byte-level entropy model, 100M parameters in the main experiments, and it is trained rather than learned jointly with BLT. Learning the patching end to end is left as future work.
- ▶ **Scaling laws are borrowed.** Training lengths follow the compute-optimal schedule derived for BPE transformers, which may set the wrong data-to-parameter ratio for a byte-level model. Most architectural ablations were run at or below 1B parameters.
- ▶ **Patch size is a new hyperparameter.** Larger patches buy inference FLOPs at some cost in accuracy, and the sequence length must be adjusted so that models with different patch sizes still see the same number of bytes.
- ▶ **Word-level manipulation stays harder.** Building words up from bytes is easier than taking them apart, which is why BPE still wins on word insertion and deletion.

Large Language Diffusion Models (LLDMs)

Large Language Diffusion Models

Shen Nie^{1*†} Fengqi Zhu^{1*†} Zebin You^{1†} Xiaolu Zhang^{2‡} Jingyang Ou¹ Jun Hu^{2‡} Jun Zhou²
Yankai Lin^{1‡} Ji-Rong Wen¹ Chongxuan Li^{1‡¶}

Abstract

Autoregressive models (ARMs) are widely regarded as the cornerstone of large language models (LLMs). We challenge this notion by introducing **LLaDA**, a diffusion model trained from scratch under the pre-training and supervised fine-tuning (SFT) paradigm. LLaDA models distributions through a forward data masking process and a reverse process, parameterized by a vanilla Transformer to predict masked tokens. By optimizing a likelihood bound, it provides a principled generative approach for probabilistic inference. Across extensive benchmarks, LLaDA demonstrates strong *scalability*, outperforming our self-constructed ARM baselines. Remarkably, LLaDA 8B is competitive with strong LLMs like LLaMA3 8B in *in-context learning* and, after SFT, exhibits impressive *instruction-following* abilities in case studies such as multi-turn dialogue. Moreover, LLaDA addresses the reversal curse, surpassing GPT-4o in a reversal poem completion task. Our findings establish diffusion models as a viable and promising alternative to ARMs, challenging the assumption that key LLM capabilities discussed above are inherently tied to ARMs. Project page and codes: <https://ml-gsai.github.io/LLaDA-demo/>.

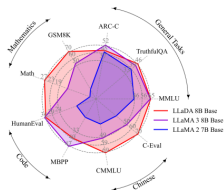


Figure 1. **Zero/Few-Shot Benchmarks.** We scale LLaDA to an unprecedented size of 8B parameters from scratch, achieving competitive performance with strong LLMs (Dubey et al., 2024).

distribution $p_{data}(\cdot)$ by optimizing a model distribution $p_{\theta}(\cdot)$ through maximum likelihood estimation, or equivalently KL divergence minimization between the two distributions:

$$\max_{\theta} \mathbb{E}_{p_{data}(x)} \log p_{\theta}(x) \Leftrightarrow \min_{\theta} \underbrace{\text{KL}(p_{data}(x) || p_{\theta}(x))}_{\text{Generative modeling principles}}. \quad (1)$$

Source: Nie et al., “Large Language Diffusion Models,” NeurIPS 2025, [arXiv:2502.09992v2](https://arxiv.org/abs/2502.09992v2), first page.

- ▶ **Emerging trend:** Use diffusion instead of autoregression for text generation

Key Idea: the model is a **mask predictor**, not a next-token predictor.

- ▶ A forward process corrupts text by replacing tokens with a mask symbol; the reverse process fills masks back in.
- ▶ The reverse process is a transformer **without a causal mask**, so every prediction is conditioned on the whole sequence in both directions.
- ▶ Generation starts from a fully masked answer and unmask tokens over repeated passes, in whatever order the model is most confident about, rather than strictly left to right.

LLaDA (Nie et al., 2025) is the first such model built at LLM scale:

- ▶ 8B parameters, pre-trained from scratch on 2.3T tokens, then supervised fine-tuned on 4.5M prompt-response pairs, using the same recipe as an autoregressive LLM.
- ▶ Competitive with LLaMA 3 8B in in-context learning on 2.3T tokens against its 15T, and ahead of it on mathematics and Chinese benchmarks.
- ▶ Answers reversal questions that left-to-right models fail, beating GPT-4o at completing a poem backwards.

Antecedents:

- ▶ Diffusion-LM (Li et al., 2022): continuous diffusion over word-vector sequences, for controllable generation at small scale.
- ▶ Masked, or discrete, diffusion: the forward process replaces tokens with a mask symbol instead of adding Gaussian noise, which is what makes text diffusion tractable at scale.

Note: Still early work, but it shows that scale, in-context learning, and instruction following are not exclusive to left-to-right models

Large Language Diffusion Models

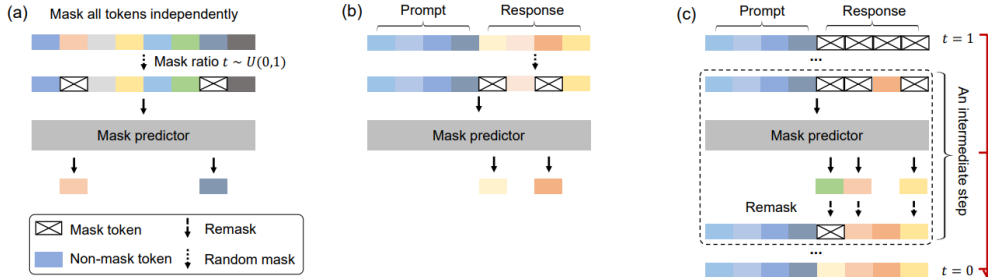


Figure 2. A Conceptual Overview of LLaDA. (a) Pre-training. LLaDA is trained on text with random masks applied independently to all tokens at the same ratio $t \sim U[0, 1]$. (b) SFT. Only response tokens are possibly masked. (c) Sampling. LLaDA simulates a diffusion process from $t = 1$ (fully masked) to $t = 0$ (unmasked), predicting all masks simultaneously at each step with flexible remask strategies.

Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](https://arxiv.org/abs/2502.09992v2), Figure 2.

Large Language Diffusion Models

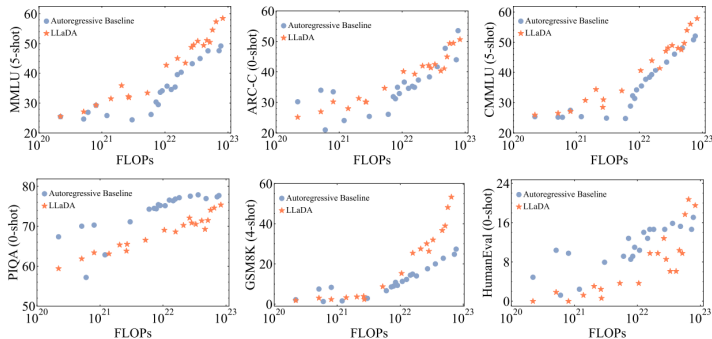


Figure 3. **Scalability of LLaDA.** We evaluate the performance of LLaDA and our ARM baselines trained on the same data across increasing computational FLOPs. LLaDA exhibits strong scalability, matching the overall performance of ARMs on six tasks.

Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](https://arxiv.org/abs/2502.09992v2), Figure 3.

LLDMs: Diffusion for Text Generation (cont.)

Large Language Diffusion Models

Table 1. Benchmark Results of Pre-trained LLMs. * indicates that LLaDA 8B Base, LLaMA2 7B Base, and LLaMA3 8B Base are evaluated under the same protocol, detailed in Appendix B.5. Results indicated by [†] and [‡] are sourced from Chu et al. (2024); Yang et al. (2024) and Bi et al. (2024) respectively. The numbers in parentheses represent the number of shots used for evaluation. "-" indicates unknown data.

	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B [†]	Qwen2.5 7B [†]	Mistral 7B [†]	Deepseek 7B [‡]
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	-	2T
General Tasks							
MMLU	65.9 (5)	65.4 (5)	45.9 (5)	70.3 (5)	74.2 (5)	64.2 (5)	48.2 (5)
BBH	49.8 (3)	57.6 (3)	37.3 (3)	62.3 (3)	70.4 (3)	56.1 (3)	39.5 (3)
ARC-C	47.9 (0)	53.1 (0)	46.3 (0)	60.6 (25)	63.7 (25)	60.0 (25)	48.1 (0)
Hellaswag	72.5 (0)	79.1 (0)	76.0 (0)	80.7 (10)	80.2 (10)	83.3 (10)	75.4 (0)
TruthfulQA	46.4 (0)	44.0 (0)	39.0 (0)	54.2 (0)	56.4 (0)	42.2 (0)	-
WinoGrande	74.8 (5)	77.3 (5)	72.5 (5)	77.0 (5)	75.9 (5)	78.4 (5)	70.5 (0)
PIQA	74.4 (0)	80.6 (0)	79.1 (0)	-	-	-	79.2 (0)
Mathematics & Science							
GSM8K	70.7 (4)	53.1 (4)	14.3 (4)	80.2 (4)	85.4 (4)	36.2 (4)	17.4 (8)
Math	27.3 (4)	15.1 (4)	3.2 (4)	43.5 (4)	49.8 (4)	10.2 (4)	6.0 (4)
GPQA	26.1 (5)	25.9 (5)	25.7 (5)	30.8 (5)	36.4 (5)	24.7 (5)	-
Code							
HumanEval	33.5 (0)	34.2 (0)	12.8 (0)	51.2 (0)	57.9 (0)	29.3 (0)	26.2 (0)
HumanEval-FIM	73.8 (2)	73.3 (2)	26.9 (2)	-	-	-	-
MBPP	38.2 (4)	47.4 (4)	18.4 (4)	64.2 (0)	74.9 (0)	51.1 (0)	39.0 (3)
Chinese							
CMMLU	69.9 (5)	50.7 (5)	32.5 (5)	83.9 (5)	-	-	47.2 (5)
C-Eval	70.5 (5)	51.7 (5)	34.0 (5)	83.2 (5)	-	-	45.0 (5)

LLaDA 8B reaches this from 2.3T training tokens, against 15T for LLaMA 3 8B. Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](https://arxiv.org/abs/2502.09992v2), Table 1 (several values were revised in v3).

Table 4. **Visualization of the Sampling Process and a Generated Multi-round Dialogue.** In the response of LLaDA, darker colors indicate tokens predicted in the later stages of sampling, while lighter colors correspond to earlier predictions.

Sampling Process	
User	Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour. How many kilometers can she run in 8 hours?
LLaDA	In 4 hours , Lily runs 12 * 4 = 48 kilometers . After 4 hours she runs 6 * 4 = 24 kilometers . In total she runs 48 + 24 = 72 kilometers in 8 hours . Result 72
Multi-round Dialogue Case	
User	Could you please share with me what the initial two lines of the renowned poem "The Road Not Taken"?
LLaDA	Certainly! The initial two lines of the renowned poem "The Road Not Taken" by Robert Frost are: "Two roads diverged in a yellow wood, And sorry I could not travel both." These lines set the stage for the poem, introducing the idea of a speaker facing a choice between two paths.
User	Please help me translate into Chinese.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into Chinese as: "两条路分岔在黄色的树林中，遗憾我不能同时走"
User	Please translate into German.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into German as: "Zwei Wege trennten sich im gelben Wald, und leider konnte ich nicht beide tun."
User	Please also write a poem about life choices. I require 5 sentences, and the first word of each sentence starts with C as the first letter.
LLaDA	Certainly! Here's a poem about life choices, with each sentence starting with the letter "C": Choices cascade through life's journey, Creating connections with each decision, Careful consideration leads to better paths, Courageous choices shape our destiny, Choosing wisely guides us through the years.

Shading marks when each token was fixed: the answer does not fill in left to right. Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](#), Table 4 (Table 3 in v3).

Forward process. A time t runs from 0, the intact sequence, to 1, a fully masked one. Draw $t \sim U(0, 1)$ and replace each token independently with the mask symbol M with probability t .

Training. The mask predictor p_θ sees the corrupted sequence x_t and predicts every masked position at once. Only masked positions contribute to the loss, rescaled by t so that heavily and lightly masked samples count equally:

$$\mathcal{L}(\theta) = -\frac{1}{tL} \sum_{i=1}^L \mathbf{1}[x_t^i = M] \log p_\theta(x_0^i | x_t) \quad (14)$$

where L is the sequence length. This is an upper bound on the negative log-likelihood, so training is still maximum likelihood, and the same objective is reused for supervised fine-tuning by masking only the response.

Sampling. Choose an answer length L and a number of steps N , and start from a fully masked answer. Each step moves from t to $s = t - 1/N$:

1. Predict a token at every masked position.
2. Remask some of those predictions, so that the number of revealed tokens grows on a fixed schedule as s falls to 0.

Which ones to remask:

- ▶ **Random remasking:** remask each fresh prediction with probability s/t .
- ▶ **Low-confidence remasking:** keep the predictions the model scores highest and remask the rest. This works better, and is why the sampling trace shown earlier fixes confident tokens first.

N is the knob that trades quality against cost: an autoregressive model needs one pass per token, while a diffusion model needs one pass per step, whatever the answer length.

Require: mask predictor p_θ , data distribution p_{data}

Pre-training. In 1% of steps the sequence length is drawn from $U[1, 4096]$ instead of the full 4096, so the model also sees short inputs. Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](#), Algorithm 1.

Algorithm 2 Supervised Fine-Tuning of LLaDA

Require: mask predictor p_θ , pair data distribution p_{data}

1: **repeat**

2: $p_0, r_0 \sim p_{\text{data}}, t \sim \text{U}(0, 1]$ # please refer to Appendix B.1 for details on the SFT data processing.

3: $r_t \sim q_{t|0}(r_t|r_0)$ # $q_{t|0}$ is defined in Eq. (7)

4: Calculate $\mathcal{L} = -\frac{1}{t \cdot L'} \sum_{i=1}^{L'} \mathbf{1}[r_t^i = \text{M}] \log p_\theta(r_0^i | p_0, r_t)$ # L' is the sequence length of r_0

5: Calculate $\nabla_\theta \mathcal{L}$ and run optimizer.

6: **until** Converged

7: **Return** p_θ

Fine-tuning. Only the response r_0 is masked; the prompt p_0 is always visible, so the same loss now trains a conditional model.

Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](https://arxiv.org/abs/2502.09992v2), Algorithm 2.

Algorithm 3 Conditional Log-likelihood Evaluation of LLaDA

Require: mask predictor p_θ , prompt p_0 , response r_0 , the number of Monte Carlo estimations n_{mc}

```
1: log_likelihood = 0
2: for  $i \leftarrow 1$  to  $n_{mc}$  do
3:    $l \sim \{1, 2, \dots, L\}$  #  $L$  is the sequence length of  $r_0$ 
4:   Obtain  $r_l$  by uniformly sampling  $l$  tokens from  $r_0$  without replacement for masking
5:    $\text{log\_likelihood} = \text{log\_likelihood} + \frac{L}{l} \sum_{i=1}^L \mathbf{1}[r_l^i = \mathbf{M}] \log p_\theta(r_0^i | p_0, r_l)$ 
6: end for
7:  $\text{log\_likelihood} = \text{log\_likelihood} / n_{mc}$ 
8: Return log_likelihood
```

Evaluation. There is no chain rule to multiply out, so likelihood is estimated by Monte Carlo: mask l random tokens, score them, and average over draws. Source: Nie et al., NeurIPS 2025, [arXiv:2502.09992v2](#), Algorithm 3.

The reported gains come with open problems:

- ▶ **Answer length is set in advance.** It is an input to sampling, not something the model decides, and an adaptive length is left to future work.
- ▶ **No inference machinery yet.** Bidirectional attention means there is no prefix to cache, so the KV-cache and the specialised attention and position schemes that autoregressive serving relies on do not carry over. The reported speed is not the point of the work.
- ▶ **Likelihoods are estimates.** Without a left-to-right factorization there is no exact sequence likelihood, only a Monte Carlo bound, which makes evaluation noisier.
- ▶ **Alignment stops at fine-tuning.** LLaDA has had no RL-based alignment, and it trails LLaMA 3 8B Instruct, which has.
- ▶ **The comparison is not fully controlled.** Compute limits kept the head-to-head against autoregressive baselines below 10^{23} FLOPs, so the largest models are compared across different training budgets.

Summary

Advancement	What it removes	What it costs
FlashAttention	The $N \times N$ attention matrix in slow GPU memory	Extra FLOPs recomputing attention in the backward pass, and kernels rewritten per GPU generation
LLaMA 3	Closed weights at the frontier	The 405B model does not fit on one 8-GPU machine in BF16
BLT	The fixed subword vocabulary	A separately trained entropy model, and patch size as a new knob
LLaDA	Left-to-right decoding	No prefix to cache, and the answer length is fixed before sampling

Open problems the four papers leave behind:

- ▶ Attention kernels co-designed with the hardware, rather than rewritten by hand for every GPU generation.
- ▶ Scaling laws derived for byte-level models, instead of borrowing the compute-optimal schedule of subword transformers.
- ▶ Patch boundaries learned end to end, rather than handed down by a separate entropy model.
- ▶ Byte-ifying existing tokenizer-based checkpoints, so a byte-level model need not be trained from scratch.
- ▶ Serving machinery for bidirectional models: caching without a prefix, and generation lengths the model chooses itself.
- ▶ RL-based alignment for diffusion language models, which so far stop at supervised fine-tuning.

- ▶ **FlashAttention:** Exact attention got fast by cutting memory traffic, not arithmetic — tiling and online softmax in the forward pass, recomputation in the backward.
- ▶ **LLaMA 3:** Open weights at frontier quality widen access to research and deployment.
- ▶ **BLT:** Learned byte patches remove the tokenizer, trading fixed vocabulary for dynamic, entropy-driven compute allocation.
- ▶ **LLDMs:** Diffusion-based generation is an alternative to left-to-right decoding, enabling parallel refinement of whole sequences.
- ▶ Each advancement removes a different fixed assumption of the standard transformer recipe — one of them inside the kernel, three at the modeling level.

References

Papers covered:

- ▶ **FlashAttention:** Dao et al., NeurIPS 2022, *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* ([arXiv:2205.14135](#))
- ▶ **Online softmax:** Milakov & Gimelshein, 2018, *Online normalizer calculation for softmax* ([arXiv:1805.02867](#))
- ▶ **Data movement:** Ivanov et al., MLSys 2021, *Data Movement Is All You Need: A Case Study on Optimizing Transformers* ([arXiv:2007.00072](#))
- ▶ **LLaMA 3:** Llama Team, AI @ Meta, 2024, *The Llama 3 Herd of Models* ([arXiv:2407.21783](#))
- ▶ **BLT:** Pagnoni et al., 2024, *Byte Latent Transformer: Patches Scale Better Than Tokens* ([arXiv:2412.09871](#))
- ▶ **LLaDA:** Nie et al., NeurIPS 2025, *Large Language Diffusion Models* ([arXiv:2502.09992](#))

Patching schemes and architectures:

- ▶ **MEGABYTE**: Yu et al., 2023, *Predicting Million-byte Sequences with Multiscale Transformers* ([arXiv:2305.07185](#))
- ▶ **SpaceByte**: Slagle, 2024, *Towards Deleting Tokenization from Large Language Modeling* ([arXiv:2404.14408](#))
- ▶ **Dynamic token pooling**: Nawrot et al., ACL 2023, *Efficient Transformers with Dynamic Token Pooling* ([arXiv:2211.09761](#))
- ▶ **Perceiver**: Jaegle et al., ICML 2021, *General Perception with Iterative Attention* ([arXiv:2103.03206](#))
- ▶ **Diffusion-LM**: Li et al., NeurIPS 2022, *Diffusion-LM Improves Controllable Text Generation* ([arXiv:2205.14217](#))

Covered in other lectures:

- **FlashAttention-2**: Dao, 2023, *Faster Attention with Better Parallelism and Work Partitioning* ([arXiv:2307.08691](https://arxiv.org/abs/2307.08691)) — see *Transformers: 2017 vs 2026*

Benchmarks:

- ▶ **CUTE**: Edman, Schmid & Fraser, EMNLP 2024, *Measuring LLMs' Understanding of Their Tokens* ([arXiv:2409.15452](https://arxiv.org/abs/2409.15452))
- ▶ **FLORES-101**: Goyal et al., *TACL* 10, 2022, pp. 522–538

Code and resources:

- ▶ FlashAttention kernels: <https://github.com/Dao-AI-Lab/flash-attention>
- ▶ Horace He, *Making Deep Learning Go Brrrr From First Principles* (operator fusion): https://horace.io/brrr_intro.html
- ▶ BLT: <https://github.com/facebookresearch/blt>
- ▶ LLaDA: <https://ml-gsai.github.io/LLaDA-demo/>
- ▶ Open LLM Leaderboard: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard